ATLAS

Voice-First Personal AI Agent

Product Requirements Document · Version 2.0

Author: Duke · NextGen Africa & Virusia Academy · April 2026

Classification: Confidential

Atlas is a cloud-brained, distributed AI agent with local workers on PC and mobile. She manages your email, calendar, finances, files, and communications autonomously — commanded through Telegram, alive 24/7 on a VPS, and extended to your devices through lightweight workers that give her eyes, hands, and a voice on every platform you use.

1. Executive Summary

Atlas is not a chatbot. She is an autonomous agent that lives in the cloud and reaches into your devices.

Atlas is a distributed, voice-and-text-first AI agent designed to serve as a private, autonomous executive assistant for a founder-operator. Unlike monolithic desktop assistants that hog resources and die when the laptop closes, Atlas is architecturally split: her brain lives permanently on a cloud VPS, her hands reach into your PC and phone through lightweight workers, and her voice reaches you through the messaging apps you already use — primarily Telegram, with WhatsApp as an outbound delivery channel.

The core design insight is that your laptop should not be the bottleneck. Atlas is always on, always reasoning, always watchful — regardless of whether your devices are powered on. When you open Telegram and say 'Atlas, what happened while I was asleep?', she already knows. She has been watching your inbox, monitoring your Paystack account, and tracking your calendar all night.

DESIGN PRINCIPLE

Speak once, get it done. Atlas is not a search engine with a voice interface. She is an agent that decomposes compound commands, executes them across multiple integrations, recovers from partial failures, and reports back with results — without hand-holding.

This document is the single source of truth for Atlas's architecture, feature set, integration plan, security model, and delivery roadmap. It covers both v1.0 (the initial production system) and v2.0 (the full autonomous agent). Every feature is tagged with its target version.

2. Vision & Strategic Goals

2.1 Vision Statement

Atlas should feel less like software and more like a capable, trusted person who happens to have access to all your accounts. She is proactive, not reactive. She surfaces information before you ask, flags problems before they escalate, and handles routine operations so completely that you forget they require human effort at all.

2.2 Primary Goals

- Deploy a 24/7 cloud-resident agent that does not depend on any personal device being powered on.

- Provide a natural, low-friction command interface through Telegram (primary) and WhatsApp (outbound).
- Achieve deep, bidirectional integration with email, calendar, finance, and the local filesystem through a distributed worker model.
- Build an extensible skill and integration architecture so new capabilities can be added without touching the core system.
- Maintain strict privacy and security — credentials never in plaintext, destructive actions always gated, all tool calls logged.
- Lay the technical foundation in v1.0 that makes v2.0's advanced autonomy a configuration change, not a rewrite.

2.3 Success Metrics

Metric	v1.0 Target	v2.0 Target	Measurement Method
Telegram command → first response	< 2.5s	< 1.5s	Automated latency logging
Task completion rate (no fallback)	> 82%	> 93%	Task log analysis
Email compose + send	< 10s	< 6s	End-to-end timing
Calendar event create + conflict check	< 8s	< 4s	End-to-end timing
Paystack balance fetch	< 3s	< 2s	API round-trip log
PC file operation (when worker online)	< 2s	< 1s	Worker job log
VPS uptime	> 99.5%	> 99.9%	UptimeRobot monitoring
False positive actions (no command)	0	0	Audit log review
Morning briefing delivery	Daily by 7am	Daily by 7am	Telegram message timestamp

3. Users & Personas

Atlas v1.0 is a single-user system, built specifically for a founder-developer profile. The architecture, permission model, and feature set are all optimised for this profile. Multi-user support is a v3 consideration and is out of scope for this document.

Attribute	Profile Detail
Name	Duke (primary user and system owner)
Role	Founder, software developer, educator — operating multiple parallel projects
Technical level	High — comfortable with VPS management, APIs, Docker, Python, Node.js
Primary devices	Laptop (primary development), Android/iPhone (mobile), VPS (always-on brain)
Communication style	Direct, fast, low-tolerance for friction or unnecessary confirmation prompts
Key pain points	Context-switching overhead, communication delays, repetitive admin, managing multiple client projects simultaneously
Privacy stance	Strong — API keys never exposed in logs, credentials in vault only, no third-party analytics
Command preference	Short, natural language commands via Telegram; voice notes acceptable for longer instructions
Working hours	Irregular — Atlas must be functional and proactive at all hours, not just 9-5

3.1 Primary Use Case Clusters

- Communications management — email, Telegram, WhatsApp on the user's behalf
- Schedule and time management — calendar, reminders, focus sessions
- Financial intelligence — Paystack monitoring, invoicing, revenue reporting
- Local machine operations — files, scripts, code, documents (via PC worker)
- Business operations — CRM, project tracking, client follow-ups
- Knowledge and research — web search, document summarisation, briefings
- Teaching automation — Virusia Academy student interactions (v1.1)
- Personal productivity — habits, routines, health logging (v2.0)

4. System Architecture

Three-tier distributed design: cloud brain, local workers, messaging interface.

Atlas is not a single application. It is a distributed system composed of three tiers that communicate over a secure, persistent WebSocket and task queue connection. Each tier has clearly defined responsibilities and can fail independently without taking down the entire system.

4.1 Architecture Overview

ARCHITECTURE DIAGRAM (TEXT)

```
[ Telegram Bot ]   [ WhatsApp (Outbound) ]                    ↕                    ↓
┌──────────────────────────────────────────┐                  │        VPS BRAIN (24/7)
│   │ Orchestrator · LLM · Memory          │   │ Email · Calendar · Paystack · Web   │
└──────────────────────────────────────────┘                                  │ Secure
WebSocket + Task Queue                  ┌──────────────────────┐        [ PC WORKER ]
[ MOBILE WORKER ]     Files · Scripts          Contacts · Location     Screenshots
Camera · SMS · Notifs
```

4.2 Tier 1 — VPS Brain

The VPS is Atlas's permanent residence. It runs 24/7, handles all intelligence and orchestration, and owns all third-party integrations that do not require local device access. This is the only tier that runs the LLM and manages memory.

Component	Responsibility	Technology
Telegram Gateway	Receive commands, send responses, manage bot state	python-telegram-bot / Telethon
WhatsApp Adapter	Outbound message delivery to contacts and clients	Baileys (WA Web) via Node.js microservice
Orchestration Engine	Intent parsing, task decomposition, tool routing, plan execution	Python + LangGraph or custom DAG
LLM Core	Reasoning, generation, tool selection, response synthesis	Groq API (speed) + Anthropic API (complex reasoning)
Memory System	Short-term, episodic, factual, procedural memory layers	ChromaDB (vector) + PostgreSQL (structured)
Email Adapter	Gmail / IMAP read, compose, send, search, categorise	Google API Client + imaplib
Calendar Adapter	Google Calendar / CalDAV full CRUD + conflict detection	Google API Client + caldav
Paystack Adapter	Transactions, balance, transfers, customer data, webhooks	Paystack REST API + httpx
Web Search Adapter	Live web search, article fetch, content summarisation	Brave Search API / SerpAPI + Trafilatura
Worker Broker	Dispatch jobs to PC/mobile workers, collect results, manage timeouts	Redis + Celery
Scheduler	Cron-style recurring tasks (morning	Celery Beat

Component	Responsibility	Technology
	briefing, follow-up reminders)	
Security Module	Credential vault, permission gates, audit log	Keyring + cryptography + append-only SQLite log
API Server	Worker registration, health checks, webhook ingestion	FastAPI

4.3 Tier 2 — PC Worker

The PC worker is a lightweight background daemon running on the user's laptop. It has no LLM, no memory, and no intelligence. It is a dumb executor that receives structured job packets from the VPS, runs them, and returns results. It should consume negligible resources when idle (< 50MB RAM, < 0.5% CPU).

Capability	Details	Technology
File read	Open and return content of text, markdown, PDF, code files	pathlib + PyMuPDF
File write / create	Write content to disk; new files or overwrite with confirmation	pathlib + confirmation gate
File search	Full-text keyword search; semantic search via local embeddings	ripgrep + sentence-transformers
File move / copy / delete	Filesystem operations with destructive-action gating	shutil + confirmation gate
Directory listing	List contents of a directory with metadata	os + pathlib
Script execution	Run shell or Python scripts; stream stdout/stderr to VPS	subprocess + asyncio
Screenshot	Capture current screen state; send to VPS for vision analysis	Pillow / mss
Application control	Open, close, or interact with desktop apps	subprocess + pyautogui
Document generation	Create PDF, DOCX, or Markdown from structured content	ReportLab + python-docx
Clipboard access	Read or write clipboard content	pyperclip
File watcher	Monitor directories for new/changed files; push events to VPS	watchdog
VPN / network status	Report current network and VPN connection state	psutil + subprocess

WORKER OFFLINE BEHAVIOUR

When the PC worker is offline (laptop closed, powered off, or worker process stopped), the VPS brain handles this gracefully. It queues any PC-bound tasks, reports their pending status to the user in Telegram, and executes them automatically when the worker reconnects. The user is notified of reconnection and queued task completion.

4.4 Tier 3 — Mobile Worker

The mobile worker is a lightweight app on the user's phone. Like the PC worker, it is a dumb executor. It gives Atlas access to phone-native capabilities that cannot be replicated from the cloud. It communicates with the VPS over a persistent WebSocket, authenticated by a device-specific token.

Capability	Details	Technology
Contacts access	Read, search, and export device contact list	Expo Contacts API
Location	Current GPS coordinates; location history (opt-in)	Expo Location
Camera	Capture photo and send to VPS for vision analysis	Expo Camera
Push notifications	Receive and display rich notifications from Atlas	Expo Notifications / FCM
SMS read (Android)	Read SMS inbox for monitoring (opt-in)	Expo SMS / native Android API
SMS send (Android)	Send SMS messages on user's behalf	Expo SMS
Local file access	Access phone storage — photos, documents, downloads	Expo FileSystem
Voice note recording	Record and send audio to VPS for transcription	Expo AV
Calendar read (phone)	Read phone calendar for cross-device conflict detection	Expo Calendar

4.5 Communication Channels

Channel	Role	Direction	Notes
Telegram Bot	Primary command interface	Bidirectional	Text, voice notes, files, inline keyboards, callback buttons
WhatsApp (Baileys)	Outbound delivery	Outbound	Sends to contacts/clients on

Channel	Role	Direction	Notes
	channel	only	user's behalf; not used for commanding Atlas
Telegram voice notes	Voice command input	Inbound	Transcribed by Whisper on VPS; treated as text command
Telegram file upload	Document/data input to Atlas	Inbound	PDFs, spreadsheets, images — processed by VPS
Telegram inline keyboards	Confirmation gates, multi-choice responses	Bidirectional	Used for all medium/high-risk confirmations
Push notification (mobile)	Proactive alerts from Atlas	Outbound	Payment received, urgent email, reminder fired
Email (outbound)	Send emails on user's behalf	Outbound	Atlas composes, user confirms via Telegram, Atlas sends

4.6 Data Flow — Standard Command

1. User sends text or voice note to Atlas Telegram bot.
2. If voice note: VPS transcribes audio using faster-whisper.
3. Orchestration engine parses intent, classifies task type, checks required tools.
4. Memory retrieval: fetch relevant episodic context and user preferences.
5. LLM generates action plan — ordered list of tool calls with inputs.
6. Executor runs action plan: VPS-local tools execute immediately; PC/mobile tasks dispatched via Redis to the appropriate worker.
7. Workers execute jobs, stream results back to VPS.
8. VPS assembles final response, synthesises narrative, sends to Telegram.
9. Full interaction (command, plan, tool calls, result) written to audit log and memory.

5. Command Interface — Telegram & WhatsApp

5.1 Telegram Bot — Primary Interface

Telegram is the primary interface through which all commands are sent to Atlas. The bot is a private bot — only the authorised user ID can interact with it. Any message from an unauthorised ID is silently ignored and logged.

5.1.1 Supported Input Types

- Text messages — natural language commands in any format or length.
- Voice notes — transcribed server-side; treated identically to text after transcription.
- Photos and images — passed to vision model for analysis; user can ask questions about them.
- Documents and PDFs — uploaded to VPS, processed, and available for reference in conversation.
- Forwarded messages — Atlas can act on forwarded emails, news articles, or messages.
- Inline bot commands — /start, /help, /status, /briefing, /stop (for aborting active tasks).

5.1.2 Confirmation UX

- Low-risk actions: Atlas executes immediately, then reports what she did.
- Medium-risk actions (send email, delete file): Atlas shows a summary with Telegram inline keyboard — [Confirm ✓] [Cancel ✗] [Edit ✎].
- High-risk actions (Paystack transfer, overwrite important file): Atlas shows a detailed review card + requires the user to type a confirmation PIN or phrase.
- Timeout: if a confirmation is not received within 5 minutes, the action is automatically cancelled and Atlas reports this.

5.1.3 Atlas Response Modes

Mode	When Used	Format
Immediate	Task < 3s, result is brief	Single Telegram message
Streaming progress	Task takes > 3s	Atlas sends 'Working on it...' then edits message with result
Multi-part	Result is long (e.g. email summary)	Multiple messages with headers; offer to send as file
File response	Result is a document, PDF, spreadsheet	Telegram file upload with brief caption
Confirmation card	Medium/high risk action	Formatted message with inline keyboard
Error report	Task failed partially or fully	Plain explanation + what succeeded + what failed + options
Proactive alert	Atlas-initiated (payment, reminder)	Telegram message with context and suggested action

5.2 WhatsApp — Outbound Delivery

WhatsApp is not a command channel. Atlas uses it exclusively to send messages on the user's behalf to contacts and clients who communicate via WhatsApp. This keeps the risk of

WhatsApp account bans minimal — Atlas is not running as a bot gateway but as an automated sender of user-initiated messages.

- Atlas can send text messages, voice notes, images, and documents to WhatsApp contacts.
- All WhatsApp sends are confirmed by the user in Telegram first.
- Contact matching: user says 'send Dana the project update on WhatsApp' — Atlas matches by name in contacts, confirms the number, then sends.
- Delivery status: Atlas reports read receipts and delivery confirmations back to Telegram.
- Virusia Academy use: Atlas sends scheduled lesson content to student WhatsApp groups on the user's behalf.

WHATSAPP RISK NOTE

WhatsApp's Terms of Service prohibit automated bulk messaging via unofficial APIs. Atlas's WhatsApp usage is intentionally low-volume and contact-targeted, not broadcast. The Baileys session runs on the user's own number and behaves like a power user, not a bot. The user accepts the small residual risk of account restriction.

6. Feature Specifications — v1.0

VERSION SCOPE

All features in Section 6 are targeted for the v1.0 release. They represent the core production system — the minimum set required for Atlas to be genuinely useful in daily operation.

6.1 Email Management ✦ v1.0

6.1.1 Supported Providers

- Gmail — full OAuth 2.0 via Google API (primary).
- Microsoft Outlook / 365 — Microsoft Graph API.
- Generic IMAP/SMTP — fallback for any other provider.

6.1.2 Core Capabilities

- Read — fetch inbox, unread count, full thread content, attachment metadata.
- Summarise — 'what did I miss in email today?' returns a structured digest by priority.
- Compose — Atlas drafts emails from a natural language description; user reviews in Telegram.
- Send — dispatched only after Telegram inline keyboard confirmation.

- Reply — Atlas drafts replies in the context of the original thread.
- Search — semantic search across email history ('find the invoice Samuel sent in February').
- Priority triage — surfaces urgent emails on each morning briefing using urgency heuristics.
- Label and categorise — apply labels/folders based on learned user patterns.
- Unsubscribe — detect marketing emails and offer one-tap unsubscribe.
- Attachment handling — download, summarise, and store attachments for reference.

6.1.3 Edge Cases

- Thread splitting — correctly links replies to parent threads even when subject lines drift.
- Large inboxes (10,000+ emails) — paginated retrieval with priority-first ordering; never loads all at once.
- Attachment > 25MB — warns user, offers to compress or link rather than attach.
- Signature management — injects correct signature based on recipient domain or contact type.
- Bounce detection — surfaces delivery failures and suggests resolution.
- Rate limiting — queues outgoing sends with exponential backoff if provider throttles.
- OAuth token expiry — proactively refreshes tokens; prompts re-auth before next use if refresh fails.
- Offline queuing — drafts composed while offline are stored locally and dispatched on reconnection.
- Ambiguous recipient — 'email John' triggers disambiguation: 'Which John — John Doe or John Smith?'

6.2 Calendar & Scheduling ✦ v1.0

6.2.1 Core Capabilities

- Create events — title, description, location, duration, attendees, recurrence, video conferencing link.
- Read — today's agenda, week view, upcoming deadlines, next available free slot.
- Update — reschedule, add/remove attendees, change location, update description by voice.
- Delete — with Telegram confirmation gate.
- Conflict detection — checks all connected calendars before creating any event.
- Smart scheduling — 'find me a free 90 minutes this week before 3pm that doesn't clash with anything.'
- Natural time parsing — 'next Thursday', 'end of the month', 'in three hours', 'the Friday after next.'
- Reminders — voice-set alerts with custom lead times; delivered via Telegram push.

- Attendee invites — generate and send calendar invitations to external attendees via email.

6.2.2 Edge Cases

- Timezone conflicts — Atlas always stores as UTC, displays in user's local timezone, converts for attendees in other zones.
- Recurring event modification — disambiguates 'this event only' vs 'all future events' before updating.
- All-day vs timed — inferred from command context; Atlas confirms interpretation before creating.
- Double-booking — warns explicitly and requires confirmation override; never silently double-books.
- External .ics invites — parsed from email attachments; Atlas surfaces accept/decline options in Telegram.
- Provider downtime — caches last known state; displays stale-data warning when calendar API is unreachable.

6.3 Financial Intelligence — Paystack ✦ v1.0

6.3.1 Core Capabilities

- Balance queries — live wallet and sub-account balances on demand.
- Transaction history — fetch by date range, amount, reference, or customer name.
- Revenue summaries — daily, weekly, monthly totals; grouped by plan, product, or customer.
- Customer lookup — find customer payment history by email or name.
- Payment alerts — proactive Telegram push notification when payment received above a configurable threshold.
- Invoice status — check whether a specific invoice has been paid, pending, or failed.
- Refund status — track refund processing state for a transaction.
- Transfers — initiate transfer to a recipient bank account (requires high-security confirmation gate — see Section 9.2).

6.3.2 Webhook Integration

- A FastAPI webhook endpoint on the VPS listens for Paystack event callbacks.
- Events handled: charge.success, transfer.success, transfer.failed, subscription.create, invoice.payment_failed.
- Each webhook event triggers the appropriate Atlas action (alert, update CRM record, send receipt).
- Webhook endpoint is authenticated with Paystack's HMAC-SHA512 signature verification.

- Exposed via Cloudflare Tunnel — no port forwarding required on VPS.

6.3.3 Edge Cases

- API key expiry — detect 401s, immediately alert user via Telegram, pause Paystack operations until key rotated.
- Duplicate webhook events — idempotency keys prevent duplicate processing.
- Pagination — handle transaction sets > 1,000 records with automatic cursor-based pagination.
- Currency — all values displayed as NGN with locale-correct formatting; secondary USD display where applicable.
- Sandbox vs live — hard configuration flag; Atlas refuses to call live API if sandbox mode is active.
- Failed transfer — surface Paystack error codes in plain English with suggested remediation steps.

6.4 Local File System Operations (via PC Worker) ✦ v1.0

6.4.1 Core Capabilities

- Read — open and return content of text, markdown, PDF, code, CSV, and JSON files.
- Write — create new files or overwrite existing with confirmation gate.
- Append — add content to the end of a file without overwriting.
- Move / copy — relocate files with path confirmation.
- Delete — moves to local trash bin first; hard delete only after second confirmation.
- Search — full-text keyword search (ripgrep); semantic similarity search via local embeddings.
- Directory listing — list contents of any scoped directory with file sizes and modified dates.
- Script execution — run shell or Python scripts; stream output back to Telegram; explain errors in plain English.
- Document generation — create PDFs, DOCX, or Markdown from dictated content.
- File watching — monitor a directory for new files; trigger Atlas actions on new arrivals (e.g. auto-summarise new PDFs in Downloads).
- Screenshot — capture screen and send to Atlas for visual analysis or record-keeping.

6.4.2 Security Boundaries

- Scoped roots — user defines allowed directories in settings; Atlas refuses access outside these.
- Hard-blocked paths — /etc, /sys, /proc, /boot, C:\Windows are permanently blocked regardless of settings.
- Sensitive file detection — warns before reading or transmitting files matching: *.pem, *id_rsa*, *.env, *secret*, *.key.

- Trash bin recovery — deleted files held in ~/.atlas-trash/ for 24 hours before permanent deletion.
- Execution sandboxing — scripts run in a subprocess with resource limits (CPU time, memory ceiling).

6.4.3 Edge Cases

- Binary files — detected and declined for read; metadata (size, type, modified date) returned instead.
- Large files (> 50MB) — chunked for summarisation; user warned of processing time before starting.
- Permission denied — OS-level permission errors surfaced in plain language with suggested fix.
- Symlink loops — circular symlink chains detected and broken; user alerted.
- Network drives / unmounted volumes — detected and reported before attempting access.
- Worker offline — PC tasks queued on VPS; executed on worker reconnect; user notified of delay.

6.5 Web Search & Research ✦ v1.0

- Live web search — query Brave Search API or SerpAPI for current information.
- Article summarisation — fetch full article content and return a structured summary.
- Multi-source synthesis — combine results from multiple sources into a single coherent answer.
- Fact-check — cross-reference a claim against multiple sources and return a confidence assessment.
- Document ingestion — user uploads a PDF or pastes a URL; Atlas reads and makes it available for Q&A.

6.6 Morning Briefing ✦ v1.0

Atlas delivers a structured morning briefing to Telegram every day at a configurable time (default: 7:00am local). The briefing is assembled autonomously from all connected integrations and requires no user prompt.

- Today's calendar — all events with times, locations, and attendees.
- Unread emails — count by category; top 3 urgent items surfaced with summaries.
- Paystack snapshot — overnight payments received, current balance, any failed transactions.
- Pending follow-ups — emails or tasks that are overdue for response.
- Key reminders — any Atlas-set reminders due today.
- Optional weather summary (if location provided).

6.7 Proactive Alerts ✦ v1.0

- Payment received — immediate Telegram push when Paystack webhook fires for a successful charge.
- Urgent email — Atlas scans new email on arrival and pushes alert if urgency score exceeds threshold.
- Calendar conflict — alert when a new event would create a clash.
- Upcoming meeting — 15-minute and 5-minute reminders for calendar events.
- Overdue follow-up — alert when an email or task has been pending longer than a configured threshold.
- PC worker offline — notify when PC worker has been disconnected for > 30 minutes.

7. Feature Specifications — v1.1

VERSION SCOPE

v1.1 features build on a stable v1.0 system. They expand Atlas's reach into developer workflows, communication platforms, and educational automation.

7.1 GitHub Integration ✦ v1.1

- Repo status — list open PRs, issues, and recent commits for a configured repo.
- PR review — fetch PR diff, summarise changes, surface review comments.
- Issue creation — create GitHub issues from a voice description.
- Commit activity — 'what did I commit yesterday?' from natural language.
- CI/CD status — check latest workflow run status; alert on failures.
- Release notes — auto-generate release notes from merged PRs since last tag.

7.2 Slack Integration ✦ v1.1

- Read channels — summarise unread messages in specified channels.
- Send messages — post to channels or DMs on user's behalf with Telegram confirmation.
- Thread replies — reply to specific threads.
- Search — semantic search across Slack history.
- Alerts — push Telegram notification when Atlas's name or configured keywords are mentioned.

7.3 Virusia Academy — Teaching Automation ✦ v1.1

Atlas manages the delivery of lesson content to Virusia Academy students on the user's behalf, freeing the user from manual scheduling and one-on-one repetitive Q&A.

- Scheduled lesson delivery — Atlas sends lesson content to student WhatsApp groups at configured times.
- Student Q&A — Atlas answers student questions using the curriculum as context; escalates complex questions to the user.
- Progress tracking — tracks which students have confirmed receipt and engagement with each lesson.
- Attendance flagging — surfaces students who have not engaged with recent content.
- Assessment reminders — sends assignment due date reminders to students.
- User briefing — daily summary of student activity delivered to user's Telegram.

7.4 Advanced Agentic Workflows ✦ v1.1

- Multi-step task chains — Atlas executes compound tasks across integrations without mid-chain prompts (e.g. 'check if Dana paid, if yes send the project files and schedule the kickoff call').
- Conditional logic — 'if I receive a payment above ₦50,000 today, send a thank-you message and log it in my CRM.'
- Scheduled workflows — define recurring multi-step routines that run automatically.
- Task recovery — if a step in a chain fails, Atlas saves state, reports the failure, and resumes from the last successful step when the issue is resolved.

8. Feature Specifications — v2.0

VERSION SCOPE

v2.0 represents Atlas's evolution into a fully autonomous agent. These features require a stable v1.x foundation and introduce significantly more complex engineering. They are planned but not yet scoped for implementation timelines.

8.1 Communications & Social Media ✦ v2.0

- Twitter / X — draft, schedule, and post tweets; monitor mentions and DMs.
- LinkedIn — draft and post professional updates; monitor connection requests and messages.

- Auto follow-up — detect emails that haven't received replies after N days; draft follow-up for user review.
- Contact enrichment — automatically update contact records with social profile data.
- Newsletter monitoring — subscribe and summarise newsletters from key contacts on a schedule.

8.2 Business CRM & Operations ✦ v2.0

- Personal CRM — maintain a database of clients, prospects, and contacts with interaction history.
- Deal pipeline — track project statuses, payment stages, and client relationship health.
- Invoice generation — auto-generate branded invoices from project data; send via email or WhatsApp.
- Proposal drafting — create project proposals from a voice description with templated sections.
- Contract tracking — monitor contract expiry dates and renewal windows.
- Client brief — 'Atlas, brief me on Dana before my call' returns recent emails, payment history, open items.
- Follow-up reminders — 'remind me to follow up with Samuel in 3 days if he hasn't replied.'

8.3 Knowledge & Research Intelligence ✦ v2.0

- Topic monitoring — Atlas watches configured keywords daily and sends a briefing when significant new content appears.
- Competitive intelligence — track what specified companies or people are publishing or announcing.
- Deep research — multi-source synthesis reports on a given topic, delivered as a structured document.
- Meeting prep — given a name or company, compile a research brief before a scheduled meeting.
- Podcast / video summarisation — transcribe and summarise audio or video content from a URL.

8.4 Developer Workflow Automation ✦ v2.0

- Test runner — run test suites on the PC worker and explain failures in plain language.
- Code scaffolding — generate boilerplate file structures from a voice description.
- Deployment assistant — trigger deployment pipelines and monitor rollout status.
- Error log analysis — watch server logs; alert on error patterns and suggest root causes.

- Dependency monitoring — alert when a project dependency has a new version or a critical CVE.
- Code review briefing — summarise open PRs across all repos and flag anything stale.

8.5 Personal Productivity & Health ✦ v2.0

- Pomodoro / focus sessions — start, track, and report on focus blocks; hold Telegram messages during deep work.
- Habit logging — log meals, exercise, sleep, or custom habits by voice; weekly summary report.
- Health reminders — medication, hydration, or supplement reminders at configurable intervals.
- Sleep intelligence — nudge user to wind down based on calendar load for tomorrow.
- End-of-day summary — what Atlas did today, what tasks are pending, tomorrow's top priorities.
- Time-blocking — Atlas proposes a daily schedule based on task list and calendar; user approves or adjusts.

8.6 Financial Intelligence — Extended ✦ v2.0

- Flutterwave integration — monitor a second payment processor alongside Paystack.
- Expense tracking — log business expenses by voice; generate monthly expense reports.
- Budget alerts — configure monthly spend ceilings per category; alert when approaching limit.
- Revenue forecasting — project next month's revenue based on historical Paystack patterns.
- Tax prep export — generate structured expense and income reports in a format suitable for accountants.

8.7 Security & System Monitoring ✦ v2.0

- Website uptime monitoring — watch configured URLs; alert on downtime within 2 minutes.
- Unusual Paystack activity — flag transactions that deviate significantly from historical patterns.
- VPS health monitoring — CPU, RAM, disk, and process alerts delivered to Telegram.
- Backup confirmation — daily confirmation that critical files and databases have been backed up.
- API key rotation reminders — alert user 14 days before any credential's configured expiry date.
- Failed login alerting — integrate with VPS auth logs; alert on repeated SSH failures.

8.8 Browser & Web Automation ✦ v2.0

- Form filling — complete web forms on behalf of the user (executed via PC worker + Playwright).
- Web scraping — extract structured data from websites on a schedule.
- Price monitoring — watch a product URL and alert when price drops below a threshold.
- Application submission — fill and submit online applications or forms with user-provided data.

8.9 Notion & Knowledge Base ✦ v2.0

- Read and write Notion pages and databases.
- Create meeting notes directly in Notion from a voice summary.
- Query Notion databases — 'what tasks are overdue in my project tracker?'
- Sync Atlas episodic memory highlights to a Notion journal page.

8.10 Multi-Agent & Atlas Network ✦ v2.0

- Specialised sub-agents — dedicated agents for research, coding, and communications that the orchestrator delegates to.
- Agent-to-agent communication — sub-agents can request data or actions from each other without user prompting.
- NextGen Africa agent — a student-facing Atlas variant that manages the academy independently.

9. Security, Privacy & Trust Model

Atlas is a cloud-first system. Security is not an afterthought — it is baked into every layer of the design.

9.1 Core Privacy Principles

- No telemetry — Atlas collects no analytics, usage data, or behavioural data for any third party.
- Credential vault — all API keys and OAuth tokens stored in the VPS keyring or an encrypted vault (Vault by HashiCorp, or a local AES-256 encrypted SQLite file). Never in .env files committed to git. Never in logs.

- Audit log — immutable, append-only log of every tool call Atlas makes, including timestamp, action taken, result, and whether user confirmation was granted. Stored in PostgreSQL with write-only access from the main process.
- Data minimisation — Atlas only fetches data it needs for the current task. No bulk data collection 'just in case'.
- User-controlled data deletion — user can instruct Atlas to wipe any or all memory categories on demand.

9.2 Confirmation Gate Matrix

Every action Atlas can take is classified by risk level. The confirmation gate defines what proof of user intent is required before execution. This cannot be overridden by a chat command — it is enforced in the executor layer.

Action	Risk Level	Gate Required
Read email / calendar / Paystack data	None	No gate — execute and report
Web search, summarise article	None	No gate — execute and report
Read local files (scoped)	None	No gate — execute and report
Draft email (not sent)	Low	No gate — Atlas shows draft for review
Create calendar event	Low	Atlas reads back details; user replies 'confirm'
Send email	Medium	Telegram inline keyboard: [Send ✓] [Edit] [Cancel]
Reply to email	Medium	Telegram inline keyboard: [Send ✓] [Edit] [Cancel]
Delete calendar event	Medium	Telegram inline keyboard + verbal 'yes delete it'
Post to social media	Medium	Telegram inline keyboard: [Post ✓] [Edit] [Cancel]
Write/overwrite local file	Medium	Telegram inline keyboard with file path shown
Delete local file	Medium	Inline keyboard + file moved to trash first
Execute local script	Medium	Atlas shows script content; user confirms via inline keyboard
Send WhatsApp message to contact	Medium	Telegram inline keyboard with recipient + message preview
Initiate Paystack transfer	High	Full transaction review card + user types 'CONFIRM TRANSFER'

Action	Risk Level	Gate Required
Permanent delete (empty trash)	High	User types 'DELETE PERMANENTLY'
Access outside scoped directories	High	Blocked by default; requires settings panel change, not chat
Modify Atlas memory or settings	Medium	Telegram inline keyboard confirmation
Add new skill or integration	Medium	Telegram inline keyboard confirmation

9.3 Telegram Security

- Allowlist-only — Atlas responds only to messages from the configured Telegram user ID. All other messages are silently dropped and logged.
- Bot token storage — Telegram bot token stored in VPS keyring only, never in source code or environment files committed to version control.
- Rate limiting — Atlas enforces a maximum of 60 commands per hour; alerts user if approaching limit due to scripted or accidental spam.
- Session hijack protection — Atlas detects abnormal patterns (commands from unusual times, unusual command sequences) and can temporarily lock itself pending user verification.

9.4 Threat Model

Threat	Likelihood	Mitigation
Telegram bot token stolen	Low	Token in keyring only; rotate immediately if compromised; allowlist by user ID provides second layer
Prompt injection via malicious email	Medium	Email content sanitised before LLM; tool calls never generated from untrusted content without review gate
VPS credential exfiltration	Low	No plaintext credentials anywhere; audit log captures all access; keyring encrypted at rest
Unintended file deletion	Low	Trash bin with 24h hold; confirmation gate; audit log for recovery
Paystack API key misuse	Low	Keys scoped to minimum permissions; rotation reminders; 401 detection pauses all Paystack ops
MITM on API calls	Very Low	All external API calls enforce TLS 1.2+; certificate pinning where API supports it
Unauthorised VPS access	Low	SSH key-only auth; fail2ban; UFW firewall; Atlas process requires separate auth token
WhatsApp account ban	Medium	Low send volume; contact-targeted only; never bulk;

Threat	Likelihood	Mitigation
		user accepts residual risk
PC worker data exfiltration	Very Low	Worker can only send data to VPS via authenticated WebSocket; no outbound access to other destinations
Memory poisoning (false memories)	Low	Memory writes validated against source; user can inspect and delete any memory record

10. Agentic Engine & Memory System

10.1 Task Planning Model

When Atlas receives a compound command, she does not execute blindly. The orchestration engine decomposes the command into an ordered action graph — a directed acyclic graph (DAG) of tool calls where each node has defined inputs, expected outputs, and a success condition. This plan is executed node by node, with results from earlier nodes fed as inputs to later ones.

- Plan preview — for multi-step tasks involving 3+ actions or any medium/high-risk action, Atlas shows the plan to the user before executing: 'Here's what I'm going to do: (1) … (2) … (3) … Confirm?'
- Partial failure handling — if a node fails, Atlas marks it, continues with independent downstream nodes, and reports the partial result at the end.
- Circuit breaker — if the same tool call fails 3 consecutive times, Atlas aborts that branch and notifies the user.
- State persistence — multi-step task state is saved to PostgreSQL after each successful node, so Atlas can resume after a crash or disconnect.

10.2 Memory Architecture

Layer	Storage	Scope	TTL	Example Content
Working memory	Python dict (in-process)	Current request only	Single request	Parsed intent, tool call chain in progress
Conversation memory	Redis	Current session	Session end + 2h	Last 20 exchanges in the conversation
Episodic memory	ChromaDB (vector)	Long-term, semantic	90 days rolling	'Invoiced Dana ₦150k on April 10', 'Samuel prefers WhatsApp'

Layer	Storage	Scope	TTL	Example Content
Factual memory	PostgreSQL	Long-term, structured	Permanent	Contact list, project names, API key metadata, user prefs
Procedural memory	YAML files	Permanent	Until edited	Skill definitions, workflow templates, briefing format
Skill cache	Redis	Hot path	1 hour	Recently used skill definitions loaded in memory

10.3 Skill System

Skills are reusable, parameterised behaviour patterns defined in YAML. They allow Atlas to execute complex repeated tasks without re-reasoning from scratch every time. Skills are the primary mechanism for the user to customise Atlas's behaviour without writing code.

- Built-in skills (v1.0) — morning briefing, inbox triage, payment alert, meeting reminder, weekly revenue summary, daily end-of-day report.
- User-defined skills — 'Atlas, whenever I say project update, send a status email to the client with the following template…'
- Skill learning — Atlas can watch a user complete a multi-step task and offer to save it as a skill: 'You do this often. Want me to save it as a skill called Project Invoice?'
- Skill invocation — triggered by keyword match, schedule, webhook event, or explicit call.
- Skill parameters — skills accept runtime parameters injected at invocation ('invoice for ₦50,000 to Dana').

11. Non-Functional Requirements

Category	Requirement	v1.0 Target	v2.0 Target
Performance	Telegram command → response start	< 2.5s	< 1.5s
Performance	Voice note transcription (60s audio)	< 8s	< 4s
Performance	PC worker round-trip (file read)	< 2s	< 1s
Performance	Paystack balance fetch	< 3s	< 2s
Reliability	VPS process uptime	> 99.5%	> 99.9%

Category	Requirement	v1.0 Target	v2.0 Target
Reliability	Auto-retry on transient API failure	3 attempts, exp. backoff	5 attempts, exp. backoff
Reliability	Task state persistence on crash	Yes — PostgreSQL	Yes — PostgreSQL + Redis snapshot
Scalability	Concurrent task execution	3 parallel tasks	10 parallel tasks
Storage	Total VPS storage footprint	< 5GB	< 10GB
Storage	Episodic memory retention	90 days	Configurable (default 180 days)
Resource (PC Worker)	Idle RAM usage	< 50MB	< 30MB
Resource (PC Worker)	Idle CPU usage	< 0.5%	< 0.2%
Security	Credential storage	VPS keyring / AES vault	HashiCorp Vault or equivalent
Security	Audit log integrity	Append-only SQLite	Append-only + daily hash chain
Extensibility	Time to add a new REST API integration	< 6 hours	< 3 hours (template-based)
Portability	OS support (VPS)	Ubuntu 22.04 LTS	Ubuntu 22.04 / Debian 12
Portability	OS support (PC Worker)	macOS, Ubuntu, Windows 11	macOS, Ubuntu, Windows 11
Observability	Structured logging	JSON logs to file	JSON logs + optional Grafana/Loki

12. Recommended Technical Stack

12.1 VPS Brain

Layer	Technology	Rationale
Runtime	Python 3.11+	Best AI/ML ecosystem; async support; rich library ecosystem
LLM (primary)	Groq API — Llama 3.3 70B /	Sub-second inference; free tier

Layer	Technology	Rationale
	Mixtral	generous; best speed/quality ratio
LLM (complex)	Anthropic API — Claude Sonnet	Best reasoning for multi-step planning; used selectively to control cost
STT	Groq Whisper API / faster-whisper	Groq Whisper for speed; faster-whisper as local fallback
Orchestration	LangGraph (state machine) or custom DAG	Structured multi-step agent with state persistence and branch handling
Vector memory	ChromaDB (embedded)	No server; persistent; fast semantic search; Python-native
Structured DB	PostgreSQL + SQLAlchemy	Reliable; ACID; good async support; handles CRM and audit log
Cache / queue broker	Redis	Task queue (Celery), session memory, rate limiting, worker heartbeats
Task queue	Celery + Celery Beat	Proven async task management; cron-style scheduling built in
API server	FastAPI	Async; fast; auto-docs; handles webhooks and worker registration
Telegram integration	python-telegram-bot v20	Async; well-maintained; full Bot API support
WhatsApp integration	Baileys (Node.js microservice)	WA Web protocol; runs as sidecar service called by FastAPI
Email	Google API Client + imaplib/smtplib	OAuth 2.0 for Gmail; IMAP fallback for others
Calendar	Google API Client + caldav	OAuth for Google Calendar; CalDAV for others
Paystack	httpx (async) + custom adapter	Official SDK lacks async; httpx with a clean adapter layer
Web search	Brave Search API / SerpAPI	Privacy-respecting; fast; structured JSON results
Web fetch	Trafilatura + httpx	Best-in-class article extraction; removes boilerplate
Credential vault	keyring + cryptography (AES-256)	OS keyring primary; encrypted file vault fallback for VPS headless
Tunnel (webhooks)	Cloudflare Tunnel (cloudflared)	No port forwarding; TLS; free; stable for webhook ingestion
Process management	systemd + supervisor	Auto-restart on crash; startup on boot; log rotation

12.2 PC Worker

Layer	Technology	Rationale
Runtime	Python 3.11+	Matches VPS stack; same libraries
Packaging	PyInstaller (single binary)	One-click install; no Python required on user machine
VPS connection	WebSocket (websockets library)	Persistent, low-latency bidirectional comms with VPS
File operations	pathlib + shutil	Cross-platform; clean API
File search	ripgrep (subprocess)	10-100x faster than Python glob for large directories
Semantic search	sentence-transformers (local)	Offline embeddings; ~500MB model; runs on CPU
Script execution	subprocess + asyncio	Async streaming of stdout/stderr back to VPS
File watching	watchdog	Cross-platform filesystem events
Screenshot	mss + Pillow	Fast cross-platform screen capture
PDF reading	PyMuPDF (fitz)	Fast; good text extraction; handles scanned PDFs
Document generation	python-docx + ReportLab	DOCX and PDF generation without LibreOffice dependency
Startup	systemd service (Linux) / LaunchAgent (macOS) / Task Scheduler (Win)	Auto-start on login; runs in background

12.3 Mobile Worker

Layer	Technology	Rationale
Framework	React Native (Expo)	Single codebase for iOS and Android; large ecosystem
VPS connection	WebSocket (expo-websocket or native)	Persistent connection; reconnect on app foreground
Background tasks	Expo TaskManager + BackgroundFetch	Limited background execution on mobile; designed around this constraint
Contacts	Expo Contacts	Cross-platform contact access with permission model
Location	Expo Location	GPS access; background location opt-in
Camera	Expo Camera	Photo capture; pass to VPS for vision analysis

Layer	Technology	Rationale
Notifications	Expo Notifications + FCM/APNs	Rich push notifications from Atlas
File system	Expo FileSystem	Access to phone storage with permission
Audio recording	Expo AV	Voice note recording for VPS transcription

13. Phased Delivery Plan

Phase	Version	Key Deliverables	Target Week
Foundation	v0.1	VPS setup, Telegram bot gateway, basic Q&A, LLM routing (Groq + Anthropic)	Week 1-2
Brain Core	v0.2	Orchestration engine, memory system (ChromaDB + PostgreSQL), audit log	Week 3-4
Communications	v0.3	Gmail full integration (read, compose, send), Google Calendar full CRUD	Week 5-6
Finance	v0.4	Paystack adapter (read), webhook ingestion, payment alerts, morning briefing	Week 7-8
PC Worker Alpha	v0.5	PC worker daemon, file read/write/search, script execution, WebSocket comms	Week 9-10
Paystack Write	v0.6	Transfer initiation with high-security gate, Paystack write operations	Week 11
WhatsApp	v0.7	WhatsApp outbound via Baileys sidecar, contact matching, delivery receipts	Week 12
Security Hardening	v0.8	Confirmation gate enforcement, credential vault, threat model audit	Week 13
Mobile Worker Alpha	v0.9	Mobile worker app, contacts, location, push notifications, voice recording	Week 14-15
v1.0 Release	v1.0	All v1.0 features stable, skill system, proactive alerts, full documentation	Week 16
Developer Tools	v1.1a	GitHub integration, Slack integration	Week 18-19
Academy	v1.1b	Virusia Academy WhatsApp automation, student Q&A, progress tracking	Week 20-21
Advanced Agent	v1.1c	Multi-step conditional workflows, scheduled workflow engine	Week 22-23
v1.1 Release	v1.1	All v1.1 features stable and tested	Week 24

Phase	Version	Key Deliverables	Target Week
CRM & Ops	v2.0a	Personal CRM, invoice generation, deal pipeline, auto follow-up	Week 28-30
Social & Research	v2.0b	Twitter/LinkedIn posting, topic monitoring, deep research, meeting prep	Week 31-33
Browser Automation	v2.0c	Playwright integration on PC worker, form filling, web scraping	Week 34-36
Health & Productivity	v2.0d	Habit logging, time-blocking, focus sessions, end-of-day reports	Week 37-38
v2.0 Release	v2.0	Full autonomous agent — all features, multi-agent architecture, Notion sync	Week 40

14. Open Questions & Decision Points

Question	Options	Decision Needed By
LLM cost management strategy	Token budget per day vs per-request routing logic vs local Ollama fallback for cheap tasks	v0.2
VPS provider and specs	Hetzner (recommended) vs DigitalOcean vs AWS Lightsail — min 4 vCPU / 8GB RAM recommended	v0.1
Paystack transfer confirmation UX	Typed passphrase vs Telegram inline PIN vs OTP via SMS — typed passphrase is simplest to implement	v0.6
WhatsApp session management	Personal number (risk of ban) vs dedicated number (costs ~₦3k/month SIM) vs WhatsApp Business API (official, higher cost)	v0.7
Mobile worker distribution	Expo Go (dev only) vs TestFlight/APK sideload vs full App Store release	v0.9
Backup strategy	VPS snapshots only vs encrypted VPS-to-S3 daily vs user-managed external backup	v0.2
Ollama local LLM fallback	Run Ollama on VPS (requires GPU instance, higher cost) vs CPU-only Llama (slow) vs API-only (simpler)	v0.2
Notion vs custom knowledge base	Notion API integration (v2.0) vs building a simple Obsidian-based knowledge base	v1.1

Question	Options	Decision Needed By
	earlier	
Student WhatsApp for Academy	Baileys on user's number (simpler) vs dedicated number (cleaner separation)	v1.1b
Atlas voice persona	Stick with Piper TTS stock voice vs invest in fine-tuned custom voice later in v2.0	v2.0

15. Edge Cases & Failure Mode Master Reference

15.1 Voice & Command Parsing

Scenario	Atlas Behaviour
Ambiguous command: 'send it'	Asks: 'Send what — the last email draft, or the FundMatch proposal?'
Multiple intents in one message	Decomposes, shows plan, confirms before executing: 'I'll do 3 things. Here's the plan…'
Unrecognised command	Responds: 'I'm not sure how to do that yet. Want me to search for instructions or log this as a feature request?'
Name disambiguation: 'email John'	Asks: 'Which John — John Doe at Efishient, or John Smith from the April call?'
Voice note — low audio quality	If confidence < 75%, Atlas replies: 'I had trouble understanding that. Can you rephrase or type it?'
Very long compound command	Atlas acknowledges receipt, shows decomposed plan, confirms before starting execution
Command contradicts a previous command	Atlas surfaces the contradiction: 'Earlier you asked me to X. This conflicts — which do you prefer?'

15.2 Agentic & Multi-Step Tasks

Scenario	Atlas Behaviour
Mid-task API failure	Saves task state, reports exact failure point, offers retry / partial result / abort
LLM context window overflow	Orchestrator summarises earlier context and continues; user notified if significant compression occurs
Dependency failure (e.g. Calendar	Continues independent steps; reports skipped step; offers to

Scenario	Atlas Behaviour
API down)	retry later
User disappears mid-confirmation	Task suspended to queue after 5min timeout; resumed on next user interaction
Infinite loop detection	Circuit breaker after 3 identical consecutive tool calls; Atlas alerts user and aborts
Tool returns unexpected data shape	Validator catches schema mismatch; Atlas reports data quality issue rather than propagating bad data
Task plan generates > 10 steps	Atlas presents summary and warns user of complexity; asks for confirmation before starting

15.3 Integration-Specific

Scenario	Atlas Behaviour
No internet on VPS	All external integrations pause; Telegram alert sent if connectivity lost > 5 min; local ops continue
OAuth token expiry	Proactive refresh 24h before expiry; if refresh fails, Telegram alert with re-auth link before next use
Paystack rate limit hit	Exponential backoff; batch remaining queries; notifies user of delay
Gmail sending limit reached	Queues remaining sends; notifies user; resumes when limit resets
PC worker disconnects mid-task	VPS saves partial state; notifies user; resumes automatically when worker reconnects
Mobile worker push notification fails	Falls back to Telegram message; logs delivery failure
Paystack webhook signature mismatch	Request rejected and logged; user alerted to potential webhook tampering
WhatsApp contact not found on device	Atlas reports: 'I couldn't find [name] in your contacts. Want me to search by phone number?'
Calendar provider returns error 503	Caches last known calendar state; displays stale-data warning on all calendar responses
Large email attachment (> 25MB)	Warns user before downloading; offers to summarise without downloading full attachment

16. Glossary

Term	Definition
Action graph / DAG	A directed acyclic graph of tool calls that represents Atlas's execution plan for a multi-step task
Agentic	Capable of autonomous multi-step task execution with tool use, planning, and error recovery
Baileys	An unofficial Node.js library that implements the WhatsApp Web protocol for programmatic messaging
Celery	A Python distributed task queue used for async job processing and scheduled tasks
ChromaDB	An open-source vector database used for semantic (embedding-based) memory storage and retrieval
Circuit breaker	A safety mechanism that aborts a failing operation after N consecutive failures to prevent infinite loops
Cloudflare Tunnel	A service that exposes local servers to the internet without port forwarding, used for webhook ingestion
Confirmation gate	A mandatory user approval step enforced at the executor layer before a high-risk action runs
Episodic memory	Long-term storage of past events and interactions, searchable by semantic meaning
LangGraph	A framework for building stateful, multi-step agent workflows as directed graphs
PC Worker	A lightweight background daemon on the user's laptop that executes filesystem and local app tasks on behalf of the VPS
Procedural memory	Stored skill definitions and workflow templates that tell Atlas how to perform recurring tasks
Scoped roots	User-defined directories that the PC worker is permitted to access; all others are blocked
Skill	A reusable, parameterised action pattern defined in YAML, triggered by keyword, schedule, or event
STT	Speech-to-Text — converts audio input (voice notes) into a text transcript for processing
TTS	Text-to-Speech — converts Atlas's text responses into spoken audio output
VPS Brain	The cloud VPS server that hosts Atlas's orchestration engine, LLM, memory, and integrations — always on
WebSocket	A persistent, bidirectional network connection used for real-time communication between the VPS and local workers
Working memory	In-process, single-request context that is discarded after each task completes

Document prepared by Duke · NextGen Africa / Virusia Academy
Atlas PRD v2.0 · April 2026 · Confidential — internal use only